Ira A. Fulton Schools of Engineering
ARIZONA STATE UNIVERSITY

SCHOOL OF COMPUTING AND AUGMENTED INTELLIGENCE
engineering.asu.edu/scai

PO Box 878809
Tempe, AZ 85287-8809
Ph: 480.965.3190
FAX: 480.965.2751

PROJECT PORTFOLIO COVER SHEET

Instructions: Students must complete all sections below. The completed form must be submitted to the Project Portfolio Canvas course with the project portfolio by the posted semester deadline.

Section I: Student Information

Last Name	First Name	ASU ID
Teefey	Maclay	1238003808
Graduation Term	**All GPAs above 3.0?**	**Date submitted**
Spring 2026	☑ Yes ☐ No	2/23/2026

Section II: Project Information

PROJECT 1:

☑ Project worth at least 30% OR ☐ Project under 30% with additional work done

Course	Semester completed	Final grade in course
CSE 572: Data Mining	2025 Fall	A-

Instructor Name Yiran Luo (has certified that the project is applicable for the portfolio)

☐ Verification of instructor approval is attached

☑ Instructor approval list has been sent to advising office

PROJECT 2:

☑ Project worth at least 30% OR ☐ Project under 30% with additional work done

Course	Semester completed	Final grade in course
CSE 511: Data Processing at Scale	2025 Fall	A+

Instructor Name Samira Ghayekhloo (has certified that the project is applicable for the portfolio)

☐ Verification of instructor approval is attached

☑ Instructor approval list has been sent to advising office

Summary of MCS Porfolio

Maclay Teefey
Arizona State University

Tempe, United States
mjteefey@asu.edu

I. Introduction

For my computer science master's program at Arizona State University, I have been able to experience all phases of the data engineering process. The projects that I have selected for this portfolio represent the challenge of analyzing and developing insights from raw data, both sensor and geospatial, at a large scale. Through my undergraduate experience, I have learned how to use well maintained labelled data to learn how differing data structures and algorithms work. Each dataset was relatively small and failed to require more than basic data cleaning steps. Once I started working in a professional environment, I learned first-hand how common it is for data to be received cobbled together and the extensive pipelines to handle mislabeled or missing data.

These two projects come from two core courses in the Big Data Systems concentration, Data Mining, CSE 572, and Data Processing At Scale, CSE 511. Data Mining, CSE 572, teaches the process of analyzing patterns in large datasets, and using machine learning techniques and models to derive insights that would have required tedious manual labor. Data Processing at Scale, CSE 511, covers the techniques to design, deploy and use large databases with an emphasis on maintaining efficient processing speeds as the data scales.

II. Project Description

A. Insulin Pump Data Mining Project

To represent the process of data mining, the course project for CSE 571, Data Mining, was to data mine sensor data spanning 4 months from a glucose sensor, and a MiniMed 670G insulin pump. The goal of the project was to take sensor data, transform the data into more summarized meal and non-meal data, and perform machine learning techniques to predict whether a meal was eaten and how much food was eaten in the meal. Using knowledge about the glucose absorption process, data intervals with carb intake and no meal eaten 2 hours after were labeled as meal data and data from two hours after the meal was eaten to the next meal were recorded as non-meal data. To analyze the sensor data using machine learning models, over 30 features were extracted from the meal and non-meal data sequences. To predict whether a sequence of data contains a meal or not, scikit-learn's RandomForestClassifier model was used with cross validation to prevent overfitting. To predict how much food was eaten in a sequence of sensor data, both the distance-based K-Means and density-based DBSCAN clustering methods were utilized.

B. Geospatial Data Processing At Scale Project

Using the knowledge learned of relational and non-relational databases, CSE 511, Data Processing At Scale, split the course project into two sections: the processing of nonrelational and relational geospatial data. The first project part was to analyze business location data stored in a NoSQL database, and to filter for businesses within a specified city or within a specified maximum distance from the user's current location. This was accomplished through 3 methods: FindBusinessBasedOnCity, FindBusinessBasedOnLocation, and distanceFunction. The second project part was analyzing New York City taxi pickup information spanning 20 years to assess the popularity of various pickup locations in the city. To analyze the taxi pickup data, a SQL database of all of the taxi data, both categorical and geospatial, and processed by two functions: HotZoneAnalysis and HotCellAnalysis. For HotZoneAnalysis, taxi pickups are categorized as part of equal size rectangular zones based on the pickup's coordinates. For HotCellAnalysis, taxi pickups are categorized into equally sized three dimensional cells based on the pickup's coordinates and the time of the pickup. These spatio-temporal cells are further analyzed for popularity using the Getis-Ord (g-star) statistic.

III. Project Results

A. Insulin Pump Data Mining Project

To verify the predictive ability of the machine learning model, the RandomForestClassifier machine learning model ran a test data set and reported that the model successfully predicted unlabeled meal data 57.58% of the time and had an F1 score of 0.57. For the K-Means and DBSCAN clusters, both unsupervised and supervised evaluation metrics are utilized. The K-Means model clusters have an SSE of 100.04, and DBSCAN model clusters have an SSE of 305.03. For both clustering types, the clusters had an entropy of 0 and purity of 1, which means that every data sequence was categorized to the correct cluster.

B. Geospatial Data Processing At Scale Project

The FindBusinessBasedOnCity and FindBusinessBasedOnLocation functions were tested using a sample business NoSQL database provided as part of the project. For FindBusinessBasedOnCity, three businesses, VinciTorio's Restaurant, Salt Creek Home, and P.croissants, were found in the city of Tempe. For FindBusinessBasedOnLocation, only the business VinciTorio's Restaurant was found for businesses that were categorized with the category "Buffets" and was 10 miles from the coordinates 33.3482589, -111.9088346. The HotZoneAnalysis and HotCellAnalysis functions were tested using a sample taxi company database provided as part of the project. The test for HotZoneAnalysis found 165 zones with more than one taxi pickup, with the most occurring at the rectangle (-73.981649, 40.764287, -73.949222, 40.800591) containing 2033 taxi pickups. The test for HotCellAnalysis found the top 50 cells in G score, with the cell (-7399,4075,15) having the highest G score.

MCS Portfolio Project Report

Maclay Teefey
CSE 572: Data Mining

Arizona State University
Tempe, Arizona, United States
mjteefey@asu.edu

***Abstract*—The processing and analysis of sensor data has been an essential piece of managing the health of diabetes patients. This report's goal is to describe the process and results of extracting and analyzing sensor data from an artificial pancreas system. The project report describes the creation of a machine learning model to predict whether a person ate a meal or didn't eat a meal in a sequence of sensor data. Two sets of data from a glucose monitor and an insulin pump are split into sequences that a person ate a meal or didn't eat a meal. Various features are created to distinguish the meal and no meal data from each other. A Random Forest Model is trained through the data fed through a pipeline of the created features and correctly predicted 57.6% of unlabeled test data. This project report contains a description of the creation of a clustering model that accurately clusters meal data into groups based on the amount of carbohydrates eaten for the patient's meal. Training sensor data calculated six ground truth bins before removing the carbohydrate information from the training data. Two clustering techniques, K-Means and Density-based spatial clustering of applications with noise (DBSCAN), were used to group the unlabeled training data using features created in the first project. The clusters created by the two clustering techniques were evaluated using the ground truth bins for the sum of squared errors, purity, and entropy. Both clustering methods had an entropy of 0.0 and a purity of 1.0, but the K-Means clusters have an SSE of 100 and the DBSCAN clusters had an SSE of 305.**

Keywords— Data Mining, Machine Learning, Clustering, Data Analytics, Data Science

I. Introduction

Wearable sensors have become a tool for around one third of Americans to track their health [1]. For adults with diabetes 1, this rate is even higher at above 48% due to the widespread adoption of Continuous Glucose Monitors (CGMs) [2]. The report's goal is to report on data mining techniques for a system that utilizes CGMs called the Medtronic 670G artificial pancreas medical control system. The Medtronic system uses two sensors: a glucose sensor, and a MiniMed 670G insulin pump. To deliver insulin to the patient, the insulin pump calculates how much insulin is required to maintain the patient's insulin level from the glucose sensor. When the CGM reading is below a certain threshold or is predicted to be less than a certain threshold, the insulin pump stops delivering insulin to the patient. The CGM sensor reports the date and time of the sensor, the glucose value (mg/dL), ISIG value, and any sensor errors. The insulin pump sensor reports the date time of the sensor, basal setting, micro bolus every 5 mins, meal intake in terms of grams of carbohydrates, meal bolus, correction bolus, correction bolus, correction factor, and other information (CGM calibration, insulin alarms, auto mode exit events, and unique codes). Both sensors record and report the sensor's information every five minutes, so there are 288 segments across a 24-hour period. The CGM sensor data used spans from 12:08 PM on November 25th, 2017, to 1:22 PM on February 12th, 2018. Insulin sensor data spans from 6:59 PM on November 24th, 2017, to 1:20 PM on February 12th, 2018. The two sensors are not synchronized to each other, and both sensors have missing segments and segments with missing values. The meal intake amount is manually input into the insulin sensor, but this can result into problems gaging the that should be provided to the patient. For example, the patient may report their meal amount after eating a meal, which can leave a large time gap between when they first started eating their meal and finish eating the meal to provide insulin. There can also be inaccuracies with how many carbohydrates are eaten with each meal. To help mitigate the meal input concerns, this report describes a machine learning model to predict whether a patient ate a meal and two clustering models to group meal intake amounts without directly referencing the meal intake amount.

II. Explanation of Solutions

A. Labeling Meal Data

To distinguish between meal data and no meal data, the sensor data must extract training meal and no meal data sets. To accomplish this, sensor data where a carb intake was recorded will be considered for extraction if there were no meals recorded from the carb intake report to two hours later. Once that condition is met, the previous 30 minutes to the meal intake and the 2 hours following the meal intake are recorded as a sequence of meal data. After a meal is recorded, two hour sequences after the sequence of meal data are recorded as no meal data if they do not have any carb intake reported in the 2-hour sequence.

B. Handling Missing Data

If only complete sequence data were utilized for this project, there would not be enough training data to proceed. To be able to have enough data points to train both machine learning models, a threshold of 20% missing data point was set. For the sequences that include 80% or more of the data points, the missing glucose level values were set using linear interpolation.

C. Meal Data Distribution

When a person eats a meal, their blood glucose curve increases logarithmically until 20 minutes after it reaches its peak. The glucose level decreases from the peak over the next two hours until it returns around the person's resting glucose level [3, Fig 1]. The prominence of the peak and the amount of time it takes to return to the resting glucose level is dependent on the amount of carbohydrates consumed. If the person does not eat a meal, their glucose level will vary across a period due to natural variations in a person's glucose level and does not represent a

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©2025 IEEE

trend [4]. To distinguish between meal and no meal data and the amount of carbohydrates eaten in a meal, various features can be created to reflect the data distribution issues.

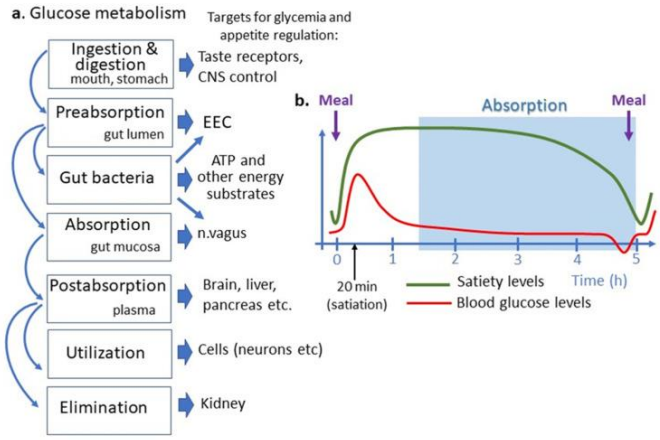

Fig. 1. Glucose metabolism process between meal times

D. Meal Data Factors

The first category of features used are basic statistical features: mean glucose level, median glucose level, standard deviation of glucose level, variance of the glucose level, minimum glucose level, maximum glucose level, the range of glucose levels, and the skew and kurtosis of the data distribution.

The second category of features used are derivative features. Due to the glucose level resembling a sine wave after eating a meal, the first and second derivatives will show a pattern resembling the velocity and acceleration curves of a sine wave if a person eats a meal. If the person did not eat a meal, the first and second derivative charts should not show a pattern and should reflect the natural variations in glucose levels across a day. For the first derivative, the ratio of increasing glucose periods, the average velocity, the standard deviation of the velocity, the maximum increase in velocity, the maximum decrease in velocity, and the average magnitude of velocity were used. For the second derivative, the ratio of accelerating glucose periods, the mean acceleration, the standard deviation of the acceleration, max acceleration, and min acceleration were utilized. Along with the derivative, the area under the curve and max consecutive increase in area under the curve were utilized as a feature.

The third category of features focuses on peak detection. The meal data should have a much more prominent peak than any other peak in the data sequence and should reach its peak around the same time as other meal sequences. Using SciPy's find peaks function, the number of peaks, the max peak height, the average peak height, the timing of the first peak, the average peak prominence, and average peak width are used as features.

The fourth category of features focuses on sine and frequency analysis. If the data is treated like a sine wave, the data should have closer frequencies and power to each other than no data. The dominant frequency, amplitude, power, and fit quality of the calculated sine wave are used as features. Using SciPy's Fast Fourier Transformation (FFT) function, the dominant frequency, power, total spectral power, spectral centroid, and spectral rolloff are extracted from the data to be utilized as features.

The final category of features tries to split the data into various sections. One set of features splits the data into 30-minute sections to better delineate the meal data distribution and non-meal data distribution. This feature extraction strategy splits the data into four 30-minute bins from 0 to 120 minutes. For each bin, the mean, median, range, max, min, and slope are calculated with all of them being individual features. Another split utilized is through splitting the data into the first half and second half. The change in slope between the first and second half, and the ratio between area under the curve for the second half and the first half were utilized as features.

E. Machine Learning Model

For the classification of meal and no meal data, the training data is run through the feature extraction pipeline and scaled using scikit-learn's StandardScalar. The model utilized for this project is scikit-learn's RandomForestClassifier with cross validation to prevent overfitting.

F. Clustering Models

For the clustering of meal data based on carbohydrate intake amounts, two models were used: K-Means and DBSCAN clustering. The meal data is split into ground truth bins of size 20 from the minimum carbohydrate intake to maximum carbohydrate intake. After calculating the ground truth bins, the truth labels were removed from the training data and the clustering models were applied. For K-Means, the model will use the best clusters from ski-learn's KMeans function using 50 different starting seeds and 1000 iterations to find each final clustering. For DBSCAN, the model uses the best clusters from ski-learn's DBSCAN function. To help DBSCAN select the correct clusters, combinations of k-distance percentiles and eps values are selected. If the number of generated density-based clusters does not match the number of ground truth bins, the highest error clusters are split using bisecting k-means until there are as many DBSCAN clusters as ground truth bins. Because the K-Means and DBSCAN clustering models are unsupervised models, the verification of clusters is more complicated than verifying the results of the supervised models. To verify that the correct ground truth bins match up with the best fitting generated cluster, Hungarian assignment is assigned to both clustering models. Once the generated clusters match the correct ground truth bins, the models can be evaluated.

III. Description of Results

A. Machine Learning Model

To verify the predictive ability of the machine learning model, the model runs a test data set and reports the metrics accuracy and F1 score. The machine learning model had an overall accuracy of 57.58% and an F1 score of 0.57.

B. Clustering Models

For the K-Means and DBSCAN models, both unsupervised and supervised evaluation metrics are utilized. SSE calculates the distance from each cluster data point to the cluster center and sums across all clusters. The K-Means model clusters have an SSE of 100.04, and DBSCAN model clusters have an SSE of 305.03. Because the ground truth bins were calculated prior to feature extraction, the purity and entropy can be calculated for

these clusters. For both the K-Means and DBSCAN models, the clusters had an entropy of 0 and purity of 1. This means that every meal data sequence was properly grouped into the correct cluster using both models.

IV. Description of Contributions

The two projects were completed individually, so this report and the code for the two projects were entirely done by me.

V. Self Reflection

To accomplish the creation of machine learning and clustering models for sensor data, multiple new skills were learned.

A. Data Processing

When analyzing data from the insulin sensor data and the CGM sensor data, issues would occur due to the insulin sensor being offset from each other by three minutes. Before this project, I had not analyzed more than one sensor csv and have not had to join two data sources more complicated than creating primary keys, so this created a good challenge.

B. Feature Extraction

For most of my previous work creating machine learning models, I have not been tasked with the creation of all the features of the model. This project forced me to learn how make multiple different categories of features to fully capture a data distribution. Due to the data distribution resembling a sine wave, I applied two different methods of finding the frequency of the data: sine wave fitting and Fourier frequency transformation.

C. Clustering

The technique of clustering through the density of points in a distribution was completely new to me prior to taking CSE 572, so I was not exposed to the necessary steps to use DBSCAN. With the clustering of DBSCAN, I have never needed to manually split my clusters through bisecting k-means, because I have not been required to force a model to extract an extra couple clusters. I have also not had to use hungarian assignment before, but it was necessary to verify and make correct the ground true bins assignment.

References

[1] *L. S. Dhingra et al., "Use of Wearable Devices in Individuals With or at Risk for Cardiovascular Disease in the US, 2019 to 2020," JAMA Netw Open, vol. 6, no. 6, p. e2316634, Jun. 2023, doi: 10.1001/jamanetworkopen.2023.16634.*

[2] D. J. DeSalvo *et al.*, "Patient Demographics and Clinical Outcomes Among Type 1 Diabetes Patients Using Continuous Glucose Monitors: Data From T1D Exchange Real-World Observational Study," *J Diabetes Sci Technol*, vol. 17, no. 2, pp. 322–328, Mar. 2023, doi: 10.1177/19322968211049783.

[3] L. V. Gromova, S. O. Fetissov, and A. A. Gruzdkov, "Mechanisms of Glucose Absorption in the Small Intestine in Health and Metabolic Diseases and Their Role in Appetite Regulation," *Nutrients*, vol. 13, no. 7, p. 2474, Jul. 2021, doi: 10.3390/nu13072474.

[4] S. Moebus, L. Göres, C. Lösch, and K.-H. Jöckel, "Impact of time since last caloric intake on blood glucose levels," *Eur J Epidemiol*, vol. 26, no. 9, pp. 719–728, Sep. 2011, doi: 10.1007/s10654-011-9608-z.

MCS Portfolio Project Report

Maclay Teefey
CSE 511: Data Processing at Scale

Arizona State University
Tempe, Arizona, United States
mjteefey@asu.edu

***Abstract*— Innovations in the storage and processing of large-scale geospatial data have enabled in-depth location-based analyses. This project report describes the data analysis methods used to process NoSQL location data and pickup location. Restaurant information stored in a NoSQL format, use city and coordinate data to find businesses within a specified city or within a specified maximum distance from the user's current location. New York City taxi information spanning years is analyzed using the coordinate data from each pickup to assess the popularity of various pickup locations in the city. The data is split into equivalently sized rectangular zones using coordinate data and equivalently sized cubic cells using the coordinate data combined with the time of the pickup. Each of the 165 zones, with at least one taxi pickup, found the total number of visits in each zone with the maximum discovered to be 2033 taxi pickups. The spatio-temporal cells are assessed for popularity using the Getis-Ord (g-star) statistic, and the highest scoring 50 cells are returned.**

Keywords— Data Processing, Geospatial Data, NoSQL, Apache Spark, Data Science

I. Introduction

Geospatial information has become an essential part of using the internet, including "check[ing] a news website or a weather app or look[ing] up directions to an appointment" [1]. The most common form of geospatial data is organized geographic information systems (GIS) and has become essential for many large-scale research projects and businesses [1]. According to Margaret Pizer, the National Science Foundation (NSF) awarded from 2021-2023 "about 180 grants, totaling more than $83 million, to support research related to GIS". This popularity has been supported by its ability to be used and stored in various formats including comma-separated values files (CSV), JavaScript Object Notation (JSON), Geographic JSON (GeoJSON), and other formats [2]. Because of the various storage formats, many companies have created various formats of NoSQL databases to store large amounts of information, including MongoDB and CosmoDB [3]. This report will cover two projects that process geospatial data in NoSQL and SQL formats. To help users find restaurants in their general area, I created two functions that will process a NoSQL database made up of restaurant information and will provide the restaurants within a specified city or within a distance limit specified by the user. To provide useful popularity data from New York City taxi pickup locations, I analyzed both coordinate data and coordinate data over time to determine which pickup locations were the most popular across the entire period or over small time periods across the dataset.

II. Explanation of Solutions

A. Processing Resturant Data

To reflect the use of NoSQL databases to store information without having a strict schema, the restaurant information uses a JSON format. Each restaurant includes the following attributes: 'buisness_id', 'type', 'state', 'latitude', 'name', 'full_address', 'categories', 'stars', 'city', 'neighborhoods', '__id', 'review_count', and 'longitude'.

B. Finding Resturants Based On Their City

To filter businesses based off a specified city, I wrote FindBusinessBasedOnCity. The function FindBusinessBasedOnCity is split into two parts: filtering the specified NoSQL collection based off of city names and then writing the filtered cities to a specified file path. To parse the NoSQL document, I used UnQLite filter function. The filter function selects which items should be included from a collection by using a function or lambda to return true if the item should be included and false if the item should not be included. To filter for businesses that should be selected because they are in the specified city, I set the lambda to only select businesses that have a 'city' JSON attribute equal to the specified city. To record the filtered businesses, I opened the file location to write specified by the user and looped through each business to record relevant information. For this function, the user needs to know the name of the business, the full address of the business, the city of the business, and the state the business is located in. For each row of the file, the businesses are stored in the format: "Name$FullAddress$City$State".

C. Finding Resturants Based On Their Location

The second task of this software was to report to businesses if they are within a specified maximum distance away and fit one of the specified lists of categories. This task required the creation of two functions: the main method, FindBusinessBasedOnLocation, and a distance calculation helper function, distanceFunction. The method FindBusinessBasedOnLocation is split into the same two parts as FindBusinessBasedOnCity: the filtering step and the writing step. Due to users expecting to find businesses in dynamic locations, the businesses' NoSQL collection does not store the distance from users as an attribute but instead stores their coordinates. These coordinates must be used to calculate the distance from the user's specified coordinates. The formulas used for distance calculation from coordinates cannot fit in one statement, so I wrote the function distanceFunction to return the distance from the business's coordinates to the user's

©2025 IEEE

coordinates. When programming distanceFunction, I assumed that the user would not need precise enough data to factor in hills and bumpiness that is not captured using a coordinate projection, so I utilized the haversine formulas to calculate the distances between two pairs of coordinates [4]. The haversine formula code, which was provided by the professor as part of this project, converts coordinate pair to radians and uses those coordinates in the radians with sine, cosine, and atan2 functions to calculate their distance apart. This distance value is checked against a maximum distance value specified by the user, so only businesses that are closer than the maximum distance is included in the filter. Once the distance from the business is calculated using distanceFunction, the business is checked to see if it fits under a list of categories provided by the user. The task of checking if a business list of categories shares a value with the specified list of categories can be accomplished in one line. By converting the business list of categories to a set, I was able to run the intersection function between the two lists, and then only include a business if the shared set between the business categories and the specified categories is not empty. Using the filter method discussed in the previous task, I set up the filter to only include businesses that pass both conditions. Once the list of businesses is filtered down to businesses that pass both conditions, I follow the same format as the previous task. However, the output written to the specified files follows the format of "Name" instead of "Name$FullAddress$City$State".

D. Processing Taxi Data

For the hot spot zone analysis project, I was provided with a sample New York City taxi company with geo-spatial data, and template code provided to load in the specific data used by the taxi company. Due to the ability to optimally load and manipulate data, I utilized Apache Spark and Scala to analyze the taxi company's data. This project was the first time I had utilized these tools, so I started by analyzing the template code in order to focus on the correct areas required to complete this project. The starter code uses Apache Spark's read, and SQL select statements to extract only the required data for each subtask. This is necessary because the data provided by the taxi company includes unnecessary data like the price of each trip. The template code is split into two mutually exclusive sections: HotZoneAnalysis and HotCellAnalysis.

E. Analyzing Hot Zones

For this project, I chose to start with the HotZoneAnalysis. The goal of the HotZoneAnalysis algorithm was to calculate the number of taxi pickups occurring in specific zones and return this information in a csv file. The HotZoneAnalysis code takes two inputs with the spark session information: pointPath and rectanglePath. The pointPath is the file location of the data for each pickup spot point, and the rectanglePath is the file location of the data for each zone, represented in rectangle form through two of the zone diagonal data points. To select all the data points in each rectangle, the HotZoneAnalysis code uses a function in HotZoneUtils called ST_Contains. ST_Contains, when a data point and a rectangle is input, can determine whether a point is contained within the rectangle. To implement this function, I split the code into three parts. Firstly, the point string and rectangle string are split using the split command on each comma in the string. The point string is split into point_x and point_y, and the rectangle string is split into corner_one_x, corner_one_y, corner_two_x, and corner_two_y values. Secondly, I use Scala's Math library to select the rectangles maximum and minimum coordinate values and assign them to variables: max_rect_x, min_rect_x, max_rect_y, and min_rect_y. Finally, I checked for each point coordinate values if it was greater than the maximum value of the coordinate or less than the minimum value of the coordinate. If either of those conditions were met for the point's x and y values, ST_Contains returned false. If none of the conditions were met, ST_Contains returned true. After selecting only, the points in a zone, I made sure that the result csv for the HotZoneAnalysis algorithm fits the required specifications by grouping points by their rectangle string and coalescing all the partitions into one file.

F. Analyzing Hot Cells

The HotCellAnalysis code determines the top 50 cells for popularity by using the G* statistic as a metric of popularity. Unlike the HotZoneAnalysis code, the HotCellAnalysis code tracks the z dimension, which corresponds to the pick-up time.

Fig. 1. Visual representation of the addition of time to create three dimensional cells from two dimensional points. [5]

The Gi* statistic formula uses neighboring cell values to calculate the popularity of certain cells, with the highest neighboring cell values resulting in the largest Gi* values. To run this formula, I created a helper function in HotCellUtils called calculateGetisOrd. This function takes in the neighbor sum, the neighbor count, the mean, standard deviation, and n value to calculate the formula (1).

$$Gi^* = (\Sigma(wij * xj) - \bar{X} * \Sigma(wij)) / (S * sqrt((n*\Sigma(wij^2) - (\Sigma(wij))^2) / (n-1))) \quad (1)$$

To use this function, I needed to calculate all these values. Firstly, I gathered all of the valid cells from the data and calculated the number of pickup points in that cell. Once I got the cell counts, I calculated the global mean, standard deviation, and n value. Secondly, I calculated the sequence of values to connect the neighboring cells. This spanned 27 cell values from (-1, -1, 1) to (1, 1, 1) and were transformed into a dataframe called offsetsDF.

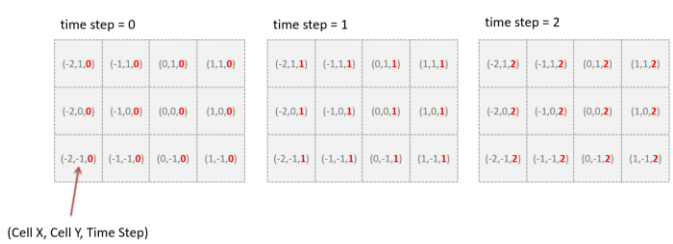

Fig. 2. Grid pattern of the combinations of cell values used in Gi-star calculation [5]

With all of these values, I joined the cell count data on the offsets dataframe by selecting values from each coordinate values added to the neighboring cell values. After gathering all of the cells with their neighboring cells, I calculated the neighborCount and neighborSums and used all of the information to add a column called gistar corresponding to the values found from running the rows in calculateGetisOrd. After each Gi* value was added, I ordered the data using gistar, and selected x, y, and z columns from the top 50 cells.

III. Description of Results

A. Resturant Analysis

The FindBusinessBasedOnCity and FindBusinessBasedOnLocation functions were tested using a sample business NoSQL database provided as part of the project. The test for FindBusinessBasedOnCity searched for businesses in the city of Tempe and wrote to output_city.txt three businesses: VinciTorio's Restaurant, Salt Creek Home, and P.croissants. The test for FindBusinessBasedOnLocation wrote to output_loc.txt only the business VinciTorio's Restaurant after searching for businesses that were categorized with the category "Buffets" and was 10 miles from the coordinates 33.3482589 latitude, -111.9088346 longitude.

B. Hot Spot Analysis

The HotZoneAnalysis and HotCellAnalysis functions were tested using a sample taxi company database provided as part of the project. The test for HotZoneAnalysis found 165 zones with more than one taxi pickup, with the most occurring at the rectangle (-73.981649, 40.764287, -73.949222, 40.800591) containing 2033 taxi pickups. The test for HotCellAnalysis found the top 50 cells in G score, with the cell (-7399,4075,15) having the highest G score.

IV. Description of Contributions

The two projects were completed individually, so this report and the code for the two projects were entirely done by me.

V. Self Reflection

To accomplish the creation of machine learning and clustering models for sensor data, multiple new skills were learned.

A. NoSQL Project

Through the process of programming this software, I learned how to process geolocation data, and furthered my understanding of processing NoSQL data. Through my previous work of processing data from the Nobel Prize API, I have learned how to process NoSQL data, as their API sent out the data using JSON. This process taught me how to navigate the various attributes in a JSON object, but I had utilized different libraries to process that data. UnQLite, a library to better process NoSQL data in Python, made the process of filtering data down to a small list of elements much easier than using loops to cycle through each data point or running the filtering through Pandas Dataframe objects. Through this filtering step, I gained more experience of using the lambda function in Python, because most of the datasets I would have used could have been done lazily through using loops to return the filtered values. This project provided my first instance of creating a distance function to calculate 3D distance. The haversine formula is an elegant form of calculating distances without having to factor in projections of the Earth or Earth's topology.

B. Hot Spot Analysis Project

This was my first time using Scala, and I was impressed by the good library functions for running joins and other large data calculations. It was also my first time using Apache Spark to package scala files into a jar file and then run the jar files using spark-submit. This process was much more difficult than it had to be for me due to the naming of the test file names switching from dash to underscore, and the inability to make spark-submit work without temporarily adding the path to the terminal window. I had previous experiences calculating and using the Getis Ord statistic, but it was useful for creating code to calculate realistic data. The calculation is much more complex when using a z dimension, and I had issues verifying my previous formula was incorrect because I didn't include the z score in the test files to see that I was clearly incorrect in my calculation. My previous formula did not include the cell itself and only included the neighbors, so I needed to script awkward joins to make the neighbor joins work well.

References

[1] M. Pizer, "Mapping science: How GIS transformed our view of the world | NSF - U.S. National Science Foundation." Accessed: Dec. 29, 2025. [Online]. Available: https://www.nsf.gov/science-matters/mapping-science-how-gis-transformed-our-view-world

[2] S. Greene and C. Rinner, "Examining the Value of Geospatial Open Data," in *The Future of Open Data*, DGO-Digital original., P. Robinson and T. Scassa, Eds., University of Ottawa Press, 2022, pp. 159–178. doi: 10.2307/jj.17610837.10.

[3] "What Is A JSON Database? | All You Need To Know," MongoDB. Accessed: Dec. 29, 2025. [Online]. Available: https://www.mongodb.com/resources/basics/databases/json-database

[4] "Calculate distance and bearing between two Latitude/Longitude points using haversine formula in JavaScript." Accessed: Dec. 28, 2025. [Online]. Available: https://www.movable-type.co.uk/scripts/latlong.html

[5] "ACM SIGSPATIAL GIS Cup 2016." Accessed: Dec. 28, 2025. [Online]. Available: https://sigspatial2016.sigspatial.org/giscup2016/problem